

Non-Dominated Points (Maxima of a Point Set)

Progress Report

Arun Dinesh Jana

Matrikel-Nr.: 2227229

Bergische Universität Wuppertal

July 9, 2026

Abstract

This report documents the progress made so far on the implementation and experimental study of the *maxima-of-a-point-set* problem (also known as computing the *non-dominated* or *Pareto-optimal* points of a set). A C++17 divide-and-conquer implementation of the MAXIMA2 algorithm and its FILTER/FILTER2 merge subroutine has been completed, following the classical treatment of the maxima problem by Preparata and Shamos. The implementation has been validated against hand-constructed test datasets and benchmarked empirically across varying input sizes. This document summarizes the problem definition, algorithmic approach, implementation structure, validation methodology, and the benchmarking results obtained to date, and outlines the parallelization work (MPI, CUDA) planned as the next stage of the project.

Contents

1	Problem Statement	2
2	Algorithmic Approach	2
2.1	Dominance Relation	2
2.2	MAXIMA2: Divide-and-Conquer Maxima	2
2.3	FILTER and FILTER2: The Merge Step	4
3	Implementation	10
3.1	Data Structures	10
3.2	Repository Structure	10
3.3	Random Data Generation	11
3.4	Build System	11
4	Validation	12
5	Benchmarking	16
5.1	Methodology	16
5.2	Results So Far	18
5.3	Planned Extensions to Benchmarking	18
6	Progress Summary	19
7	Future Scope	19

1 Problem Statement

Given a set S of N points in E^d (i.e., d -dimensional Euclidean space with integer coordinates), a point p *dominates* a point q if p is greater than or equal to q in every coordinate, with strict inequality in at least one coordinate:

$$p \succ q \iff (\forall i \in \{1, \dots, d\} : p_i \geq q_i) \wedge (\exists i \in \{1, \dots, d\} : p_i > q_i). \quad (1)$$

A point is a *maximum* (equivalently, *non-dominated* or *Pareto-optimal*) point of S if no other point in S dominates it. The *maxima problem* asks for the set of all such points. This is a classical problem in computational geometry with applications in multi-objective optimization, database skyline queries, and decision theory.

2 Algorithmic Approach

The project implements the divide-and-conquer MAXIMA2 algorithm together with its recursive FILTER merge step, as described in Preparata and Shamos, *Computational Geometry: An Introduction*.

2.1 Dominance Relation

The dominance predicate defined above is implemented directly as a coordinate-wise comparison over the first d coordinates of two points.

Listing 1: Dominance predicate

```
1 #include "dominance.hpp"
2
3 bool Dominates(const Item& a, const Item& b, size_t d)
4 {
5     bool strict = false;
6
7     for (size_t i=0; i < d; i++)
8     {
9         if(a.coords[i] < b.coords[i])
10         {
11             return false;
12         }
13
14         if(a.coords[i] > b.coords[i])
15         {
16             strict = true;
17         }
18     }
19
20     return strict;
21 }
```

2.2 Maxima2: Divide-and-Conquer Maxima

The algorithm proceeds as follows:

1. Sort the point set by its last coordinate.
2. Split the sorted set at the median, adjusting the split point so that all points tied on the last coordinate fall on the same side (this preserves correctness of the recursive step).

3. Recursively compute the maxima of the “lower” half S_1 and the “upper” half S_2 , obtaining M_1 and M_2 respectively.
4. **Merge**: every point of M_2 is automatically a maximum of S . A point of M_1 survives only if it is not dominated, on the remaining $d - 1$ coordinates, by any point of M_2 — this check is delegated to FILTER.
5. If the recursion bottoms out with all points tied on the last coordinate (i.e., the median split fails to separate the set), the maxima are computed directly by brute-force pairwise comparison.

Listing 2: MAXIMA2 divide-and-conquer driver

```

1  #include "maxima2.hpp"
2  #include "sorting.hpp"
3  #include "filter.hpp"
4  #include "dominance.hpp"
5
6
7  static std::vector<Item> BruteForceMaxima(
8      const std::vector<Item>& points, size_t d)
9  {
10     std::vector<Item> result;
11
12     for(size_t i = 0; i < points.size(); i++)
13     {
14         bool dominated = false;
15
16         for(size_t j = 0; j < points.size(); j++)
17         {
18             if(i == j) continue;
19
20             if(Dominates(points[j], points[i], d))
21             {
22                 dominated = true;
23                 break;
24             }
25         }
26
27         if(!dominated)
28         {
29             result.push_back(points[i]);
30         }
31     }
32
33     return result;
34 }
35
36 std::vector<Item> MAXIMA2(std::vector<Item> points, size_t d)
37 {
38     SortByLastCoordinate(points);
39
40     if(points.size() <= 1)
41     {
42         return points;
43     }
44
45     size_t mid = points.size()/2;

```

```

46
47     while(mid < points.size() &&
48           points[mid].coords[d-1] == points[mid-1].coords[d-1])
49     {
50         mid++;
51     }
52
53     if(mid == points.size())
54     {
55         return BruteForceMaxima(points, d);
56     }
57
58     std::vector<Item> S1(points.begin(), points.begin()+mid);
59
60     std::vector<Item> S2(points.begin()+mid, points.end());
61
62     auto M1 = MAXIMA2(S1,d);
63     auto M2 = MAXIMA2(S2,d);
64
65     if(d == 1)
66     {
67         return M2;
68     }
69
70     auto survivors = FILTER(M1, M2, d - 1);
71
72     survivors.insert(survivors.end(), M2.begin(), M2.end());
73     return survivors;
74 }

```

2.3 Filter and Filter2: The Merge Step

$\text{FILTER}(U, V, d)$ computes the subset of U that is not dominated by any point of V , considering only the first d coordinates. The general recursive case works by:

- Equipartitioning V on coordinate d into V_1 (lower half) and V_2 (upper half), via PARTITIONV .
- Deriving a splitting threshold from the maximum d -th coordinate in V_1 , via THRESHOLDFROMV1 .
- Splitting U at that threshold into U_1 and U_2 , via SPLITU .
- Recursing on three sub-problems — (U_2, V_2, d) , (U_1, V_1, d) , and $(U_1, V_2, d - 1)$ — and recombining the results via set intersection (INTERSECT).

The recursion bottoms out at $d = 1$ (a simple coordinate threshold test) and at $d = 2$, where FILTER2 performs a linear-time plane sweep over the remaining two coordinates.

Listing 3: FILTER and FILTER2

```

1 #include "filter.hpp"
2 #include "dominance.hpp"
3 #include "partition.hpp"
4 #include "threshold.hpp"
5 #include "split_u.hpp"
6 #include "intersection.hpp"
7 #include <limits>

```

```

8  #include <algorithm>
9  #include "debug.hpp"
10
11 std::vector<Item> FILTER2(
12     const std::vector<Item>& U,
13     const std::vector<Item>& V
14 )
15 {
16     std::vector<Item> Us = U;
17     std::vector<Item> Vs = V;
18
19     SortByCoordinate(Us, 1);
20     SortByCoordinate(Vs, 1);
21     std::reverse(Us.begin(), Us.end());
22     std::reverse(Vs.begin(), Vs.end());
23
24     std::vector<Item> survivors;
25
26     size_t j = 0; // next Vs index to absorb
27
28     int bestX_strict = std::numeric_limits<int>::min();
29
30     size_t i = 0;
31
32     while(i < Us.size())
33     {
34         int y = Us[i].coords[1];
35
36         // Absorb every v with strictly greater y into bestX_strict.
37         while(j < Vs.size() && Vs[j].coords[1] > y)
38         {
39             bestX_strict = std::max(bestX_strict, Vs[j].coords[0]);
40             j++;
41         }
42
43         int bestX_eq = std::numeric_limits<int>::min();
44
45         size_t k = j;
46
47         while(k < Vs.size() && Vs[k].coords[1] == y)
48         {
49             bestX_eq = std::max(bestX_eq, Vs[k].coords[0]);
50             k++;
51         }
52
53         // Process every u tied at this same y using the same two
54         // thresholds (bestX_strict, bestX_eq).
55         while(i < Us.size() && Us[i].coords[1] == y)
56         {
57             bool dominated =
58                 (bestX_strict >= Us[i].coords[0]) ||
59                 (bestX_eq > Us[i].coords[0]);
60
61             if(!dominated)
62             {
63                 survivors.push_back(Us[i]);
64             }
65

```

```

66         i++;
67     }
68
69     bestX_strict = std::max(bestX_strict, bestX_eq);
70
71     j = k;
72 }
73
74     return survivors;
75 }
76
77 std::vector<Item> FILTER(const std::vector<Item>& U, const std::vector<
78 Item>& V, size_t d)
79 {
80     DEBUG_PRINT("FILTER("<< "d=" << d << ", U=" << U.size() << ", V=" <<
81     V.size()<< ")");
82
83     if(U.empty())
84     {
85         return {};
86     }
87
88     if(V.empty())
89     {
90         return U;
91     }
92
93     if(V.size() == 1)
94     {
95         const Item& v = V[0];
96
97         std::vector<Item> survivors;
98
99         for(const auto& u : U)
100         {
101             if(!Dominates(v, u, d))
102             {
103                 survivors.push_back(u);
104             }
105         }
106
107         return survivors;
108     }
109
110     if(d == 1)
111     {
112         // Single coordinate: u survives unless some v has coords[0] > u
113         // .coords[0]
114         // (a strict improvement -- an exact tie does not dominate).
115         int maxV = V[0].coords[0];
116
117         for(const auto& v : V)
118         {
119             if(v.coords[0] > maxV)
120             {
121                 maxV = v.coords[0];
122             }
123         }
124     }

```

```

121         std::vector<Item> survivors;
122
123         for(const auto& u : U)
124         {
125             if(u.coords[0] >= maxV)
126             {
127                 survivors.push_back(u);
128             }
129         }
130
131         return survivors;
132     }
133
134     if(d == 2)
135     {
136         DEBUG_PRINT("Reachead FILTER2 " << "U=" << U.size() << " V=" <<
137                     V.size());
138
139         return FILTER2(U,V);
140     }
141
142     auto [V1,V2] = PartitionV(V,d);
143
144     DEBUG_PRINT("PartitionV -> " << "V1=" << V1.size() << " V2=" << V2.
145                 size());
146
147     int threshold = ThresholdFromV1(V1,d);
148
149     auto [U1,U2] = SplitU(U,threshold,d);
150
151     DEBUG_PRINT("SplitU -> " << "U1=" << U1.size() << " U2=" << U2.size())
152         ;
153
154     std::vector<Item> A;
155     std::vector<Item> B;
156     std::vector<Item> C;
157
158     if (!U2.empty() && !V2.empty())
159     {
160         A = FILTER(U2, V2, d);
161     }
162     else
163     {
164         A = U2;
165     }
166
167     if (!U1.empty() && !V1.empty())
168     {
169         B = FILTER(U1, V1, d);
170     }
171     else
172     {
173         B = U1;
174     }
175
176     if (!U1.empty() && !V2.empty())
177     {
178         C = FILTER(U1, V2, d - 1);
179     }
180     else
181     {
182         C = U1;
183     }
184
185     auto BC = Intersect(B,C);
186
187     A.insert(A.end(), BC.begin(), BC.end());
188
189     return A;

```

176 }

Listing 4: Equipartition of V

```
1 #include "partition.hpp"
2 #include <algorithm>
3
4 std::pair<std::vector<Item>, std::vector<Item>>
5     PartitionV(std::vector<Item> V, size_t d)
6 {
7     SortByCoordinate(V, d - 1);
8
9     size_t mid = V.size()/2;
10
11     std::vector<Item> V1(V.begin(), V.begin()+mid);
12
13     std::vector<Item> V2(V.begin()+mid, V.end());
14
15     return {V1, V2};
16 }
```

Listing 5: Threshold extraction from V_1

```
1 #include "threshold.hpp"
2
3 int ThresholdFromV1(
4     const std::vector<Item>& V1,
5     size_t d
6 )
7 {
8     int xmax = V1[0].coords[d-1];
9
10    for(const auto& p : V1)
11    {
12        if(p.coords[d-1] > xmax)
13        {
14            xmax = p.coords[d-1];
15        }
16    }
17
18    return xmax;
19 }
```

Listing 6: Threshold-based split of U

```
1 #include "split_u.hpp"
2
3 std::pair<
4     std::vector<Item>,
5     std::vector<Item>
6 >
7 SplitU(
8     const std::vector<Item>& U,
9     int threshold,
10    size_t d
11 )
12 {
13     std::vector<Item> U1;
14     std::vector<Item> U2;
```



```

15
16     for(const auto& p : U)
17     {
18         if(p.coords[d-1] <= threshold)
19         {
20             U1.push_back(p);
21         }
22         else
23         {
24             U2.push_back(p);
25         }
26     }
27
28     return {U1,U2};
29 }

```

Listing 7: Set intersection helper

```

1  #include "intersection.hpp"
2
3  static bool Equal(
4      const Item& a,
5      const Item& b
6  )
7  {
8      return a.coords == b.coords;
9  }
10
11 std::vector<Item> Intersect(
12     const std::vector<Item>& A,
13     const std::vector<Item>& B
14 )
15 {
16     std::vector<Item> result;
17
18     for(const auto& a : A)
19     {
20         for(const auto& b : B)
21         {
22             if(Equal(a,b))
23             {
24                 result.push_back(a);
25                 break;
26             }
27         }
28     }
29
30     return result;
31 }

```

Listing 8: Coordinate-based sorting helpers

```

1  #include "sorting.hpp"
2  #include <algorithm>
3
4  void SortByLastCoordinate(std::vector<Item>& items)
5  {
6      std::sort(items.begin(), items.end(),
7          [](const Item& a, const Item& b)

```

```

8         {
9             return a.coords.back() < b.coords.back();
10        }
11    );
12 }
13
14 void SortByCoordinate(std::vector<Item>& items, size_t k)
15 {
16     std::sort(items.begin(), items.end(),
17               [k](const Item& a, const Item& b)
18               {
19                 return a.coords[k] < b.coords[k];
20             }
21     );
22 }

```

3 Implementation

The implementation is written in C++17 and organized as a small modular library plus two executables built with CMake (≥ 3.10).

3.1 Data Structures

Points are represented by a simple aggregate:

Listing 9: Point representation

```

1 #pragma once
2 #include <vector>
3
4 struct Item {
5     std::vector<int> coords; // coords[0]..coords[d-1]
6 };

```

3.2 Repository Structure

```

non-dominated-points/
|-- src/
|   |-- main.cpp           (benchmark driver)
|   |-- generate_data.cpp   (random point generation)
|   |-- dominance.cpp
|   |-- sorting.cpp
|   |-- maxima2.cpp
|   |-- filter.cpp
|   |-- partition.cpp
|   |-- split_u.cpp
|   |-- threshold.cpp
|   |-- intersection.cpp
|   '-- print.cpp
|-- include/                (corresponding headers)
|-- validation/
|   |-- validator.cpp / validator.hpp
|   |-- validation_main.cpp
|   '-- test_datasets/      (Test1.txt .. Test13.txt)

```

```

|-- plots/
|   |-- plot_results.py
|   '-- figures/
|-- results/
|   '-- benchmark_results.csv
'-- CMakeLists.txt

```

3.3 Random Data Generation

Benchmark inputs are generated by drawing n points with D integer coordinates, each independently uniform on $[1, 100]$.

Listing 10: Random point generation

```

1 #include "generate_data.hpp"
2 #include <random>
3
4 using namespace std;
5
6 vector<Item> generateData(size_t n ,size_t D)
7 {
8     vector<Item> items;
9
10    random_device rd;
11    mt19937 gen(rd());
12
13    uniform_int_distribution<> range(1,100);
14
15    for (size_t i = 0; i < n; i++)
16    {
17        Item item;
18
19        for (size_t j = 0; j < D; j++)
20        {
21            item.coords.push_back(range(gen));
22        }
23
24        items.push_back(item);
25    }
26
27    return items;
28 };
29

```

3.4 Build System

The project is built with CMake and produces two targets: the benchmarking executable `non_dominated_points` and the correctness-checking executable `validator`.

Listing 11: CMakeLists.txt

```

1 cmake_minimum_required(VERSION 3.10)
2
3 project(NonDominatedPoints)
4
5 set(CMAKE_CXX_STANDARD 17)
6
7 include_directories(include)

```

```

8
9  add_executable(
10     non_dominated_points
11     src/main.cpp
12     src/generate_data.cpp
13     src/print.cpp
14     src/sorting.cpp
15     src/dominance.cpp
16     src/maxima2.cpp
17     src/partition.cpp
18     src/threshold.cpp
19     src/split_u.cpp
20     src/intersection.cpp
21     src/filter.cpp
22 )
23
24 add_executable(
25     validator
26
27     validation/validation_main.cpp
28     validation/validator.cpp
29
30     src/print.cpp
31     src/maxima2.cpp
32     src/filter.cpp
33     src/dominance.cpp
34     src/sorting.cpp
35     src/partition.cpp
36     src/threshold.cpp
37     src/split_u.cpp
38     src/intersection.cpp
39 )

```

4 Validation

Correctness of MAXIMA2 is checked against 13 hand-constructed test datasets stored in `validation/test_datasets`. Each dataset file specifies the dimension, the input point set, and the expected set of maxima, e.g.:

```

# Dimension
2

# Number of Input Points
1

# Input Points
5 5

# Number of Expected Maxima
1

# Expected Maxima
5 5

```

The dataset loader parses this format and returns the input points alongside the expected output for comparison.

Listing 12: Test dataset loader

```
1  #include "validator.hpp"
2
3  #include <fstream>
4  #include <sstream>
5  #include <iostream>
6
7  ValidationDataset LoadDataset(const std::string& filename)
8  {
9      ValidationDataset dataset;
10
11      std::ifstream file(filename);
12
13      if (!file.is_open())
14      {
15          std::cerr << "Error opening file: "
16                  << filename << std::endl;
17          return dataset;
18      }
19
20      std::string line;
21
22      // ----- Dimension -----
23      while (std::getline(file, line))
24      {
25          if (!line.empty() && line[0] != '#')
26          {
27              dataset.dimension = std::stoul(line);
28              break;
29          }
30      }
31
32      // ----- Number of Input Points -----
33      size_t inputCount = 0;
34
35      while (std::getline(file, line))
36      {
37          if (!line.empty() && line[0] != '#')
38          {
39              inputCount = std::stoul(line);
40              break;
41          }
42      }
43
44      // ----- Input Points -----
45      for (size_t i = 0; i < inputCount;)
46      {
47          std::getline(file, line);
48
49          if (line.empty() || line[0] == '#')
50              continue;
51
52          std::stringstream ss(line);
53
54          Item item;
```

```

55
56     int value;
57
58     while (ss >> value)
59     {
60         item.coords.push_back(value);
61     }
62
63     dataset.inputPoints.push_back(item);
64
65     i++;
66 }
67
68 // ----- Number of Expected Maxima -----
69 size_t expectedCount = 0;
70
71 while (std::getline(file, line))
72 {
73     if (!line.empty() && line[0] != '#')
74     {
75         expectedCount = std::stoul(line);
76         break;
77     }
78 }
79
80 // ----- Expected Maxima -----
81 for (size_t i = 0; i < expectedCount;)
82 {
83     std::getline(file, line);
84
85     if (line.empty() || line[0] == '#')
86         continue;
87
88     std::stringstream ss(line);
89
90     Item item;
91
92     int value;
93
94     while (ss >> value)
95     {
96         item.coords.push_back(value);
97     }
98
99     dataset.expectedMaxima.push_back(item);
100
101     i++;
102 }
103
104 return dataset;
105 }

```

Listing 13: Header for validation types

```

1 #pragma once
2
3 #include <vector>
4 #include <string>
5

```

```

6  #include "item.hpp"
7
8  struct ValidationDataset
9  {
10     size_t dimension;
11     std::vector<Item> inputPoints;
12     std::vector<Item> expectedMaxima;
13 };
14
15 ValidationDataset LoadDataset(const std::string& filename);
16
17 bool CompareMaxima(const std::vector<Item>& computed, const std::vector<
    Item>& expected);

```

The validation driver runs MAXIMA2 on all 13 test cases and reports a pass/fail summary, printing the expected and computed maxima side by side whenever a mismatch occurs.

Listing 14: Validation driver

```

1  #include <iostream>
2
3  #include "validator.hpp"
4  #include "maxima2.hpp"
5  #include "print.hpp"
6
7  using namespace std;
8
9  bool CompareMaxima(const std::vector<Item>& computed, const std::vector<
    Item>& expected)
10 {
11     if (computed.size() != expected.size())
12     {
13         return false;
14     }
15
16     for (size_t i = 0; i < computed.size(); i++)
17     {
18         if (computed[i].coords != expected[i].coords)
19             return false;
20     }
21
22     return true;
23 }
24
25 int main()
26 {
27     int passed = 0;
28     for (int i = 1; i <= 13; i++)
29     {
30         ValidationDataset dataset = LoadDataset("../validation/
            test_datasets/Test" + std::to_string(i) + ".txt");
31
32         auto maxima = MAXIMA2(dataset.inputPoints, dataset.dimension);
33
34         if (CompareMaxima(maxima, dataset.expectedMaxima))
35         {
36             cout << "Test Case " << i << " : PASSED\n\n";
37             passed++;
38         }

```

```

39     else
40     {
41         cout << "Test Case " << i << " : FAILED\n";
42
43         cout << "\nExpected Maxima\n";
44         printItems(dataset.expectedMaxima);
45
46         cout << "\nComputed Maxima\n";
47         printItems(maxima);
48
49         cout << "\n";
50     }
51 }
52 cout << "\n===== \n";
53 cout << "Validation Summary\n";
54 cout << "===== \n";
55 cout << "Passed : " << passed << " / 13\n";
56
57
58 return 0;
59 }

```

At the current stage, all 13 test cases pass against the implemented MAXIMA2/FILTER logic.

5 Benchmarking

5.1 Methodology

The benchmark driver (`src/main.cpp`) generates random point sets and measures the wall-clock execution time of MAXIMA2 using `std::chrono::high_resolution_clock`, averaging over 3 repetitions per configuration. Results are written to `results/benchmark_results.csv` and subsequently plotted by `plots/plot_results.py` (using pandas and matplotlib) into `plots/figures/`.

Listing 15: Benchmark driver

```

1  #include <iostream>
2  #include <vector>
3  #include "item.hpp"
4  #include "generate_data.hpp"
5  #include "print.hpp"
6  #include "maxima2.hpp"
7  #include <chrono>
8  #include <fstream>
9
10 using namespace std;
11
12 int main()
13 {
14     //size_t Dim = 10, NPoints = 1000;
15
16     const vector<size_t> dims = {10};
17
18     const vector<size_t> npoints = {100, 500, 1000, 5000, 10000, 50000,
19                                     100000};
20
21     ofstream csv_file("../results/benchmark_results.csv");
22     csv_file << "Dimension,NPoints,ExecutionTime_us\n";

```



```

23   for (size_t Dim : dims)
24   {
25       for (size_t NPoints : npoints)
26       {
27           //repeat times and take average time
28           vector<double> execution_times;
29           const int num_iterations = 3;
30
31           vector<Item> Originalitems = generateData(NPoints, Dim);
32
33           for (int i = 0; i < num_iterations; ++i)
34           {
35               vector<Item> items = Originalitems; // Copies the
36               // original items for each iteration
37
38               auto start = chrono::high_resolution_clock::now();
39
40               auto maxima = MAXIMA2(items, Dim);
41
42               auto stop = chrono::high_resolution_clock::now();
43
44               double elapsed = chrono::duration<double, micro>(stop -
45               start).count();
46
47               execution_times.push_back(elapsed);
48           }
49
50           // Calculate average time
51           double average_time = 0.0;
52           for (double t : execution_times)
53           {
54               average_time += t;
55           }
56           average_time /= execution_times.size();
57           csv_file << Dim << "," << NPoints << "," << average_time <<
58           "\n";
59       }
60   }
61   csv_file.close();
62   cout << "Benchmark results saved to ../results/benchmark_results.csv
63   \n";
64   cout << "\nGenerating Plots...\n";
65
66   int status = system("python3 ../plots/plot_results.py");
67
68   if (status == 0)
69   {
70       cout << "Plots generated successfully.\n";
71   }
72   else
73   {
74       cout << "Error generating plots.\n";
75   }
76   return 0;
77 };

```

Listing 16: Plotting script

```

1 from pathlib import Path
2 import pandas as pd
3 import matplotlib.pyplot as plt
4
5 BASE_DIR = Path(__file__).resolve().parent
6 PROJECT_DIR = BASE_DIR.parent
7
8 RESULTS_DIR = PROJECT_DIR / "results"
9 FIGURES_DIR = BASE_DIR / "figures"
10
11 FIGURES_DIR.mkdir(exist_ok=True)
12
13 # Read benchmark data
14 df = pd.read_csv(REULTS_DIR / "benchmark_results.csv")
15
16 # Create one plot for each dimension
17 for dim in sorted(df["Dimension"].unique()):
18
19     subset = df[df["Dimension"] == dim].sort_values("NPoints")
20
21     plt.figure(figsize=(8, 5))
22
23     plt.plot(
24         subset["NPoints"],
25         subset["ExecutionTime_us"] / 1000, # Convert to milliseconds
26         marker="o",
27         linewidth=2,
28         markersize=6
29     )
30
31     plt.xscale("log")
32     plt.yscale("log")
33     plt.xlabel("Number of Points")
34     plt.ylabel("Execution Time (ms)")
35     plt.title(f"Runtime vs Number of Points for ({dim}D)")
36     plt.grid(True)
37
38     plt.tight_layout()
39
40     plt.savefig(
41         FIGURES_DIR / f"runtime_{dim}D.png",
42         dpi=300
43     )
44
45     plt.close()
46
47 print("Plots saved successfully!")

```

5.2 Results So Far

To date, benchmarks have been run for a single fixed dimension, $D = 10$, over input sizes ranging from $N = 100$ to $N = 100,000$ points. The raw measurements are reproduced in Table 1.

5.3 Planned Extensions to Benchmarking

The benchmark driver currently loops over a single dimension list (`dims = {10}`); the code already supports sweeping over multiple dimensions (as originally sketched, $D = 2 \dots 10$), and

Table 1: Average execution time of MAXIMA2 for $D = 10$ (averaged over 3 runs per configuration).

N (points)	Execution time (μs)
100	11,692.4
500	70,702.9
1,000	193,519
5,000	1,873,410
10,000	4,995,070
50,000	37,591,500
100,000	79,678,700

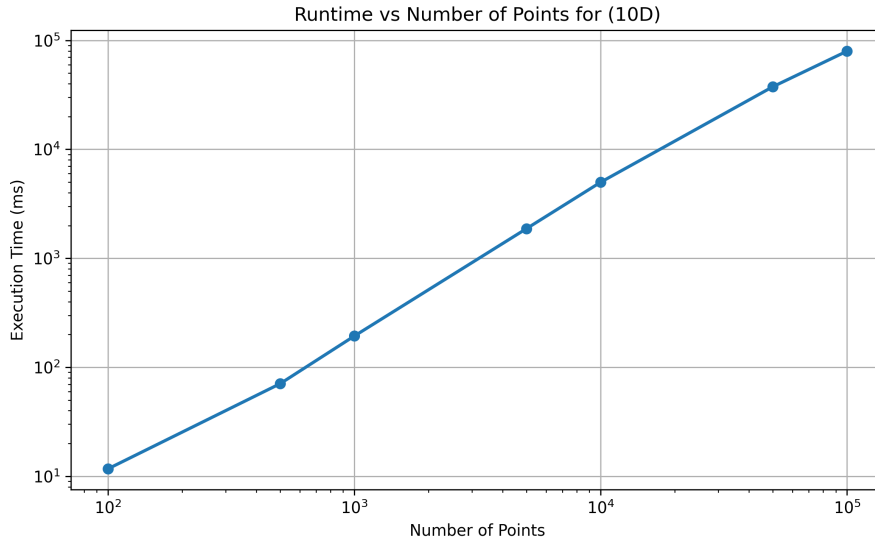


Figure 1: Execution time of MAXIMA2 versus number of points, for $D = 10$ (log-log scale).

this is planned as an immediate next step so that runtime can be compared across both N and d .

6 Progress Summary

- **Completed:** dominance relation, random data generation, MAXIMA2 divide-and-conquer driver (with brute-force fallback), full recursive FILTER/FILTER2 merge step, CMake build system with two targets, correctness validation against 13 hand-crafted test cases, and an initial benchmarking pipeline with automated plot generation.
- **In progress / next steps:** broaden benchmarking to multiple dimensions ($D = 2, \dots, 10$) and larger repetition counts for statistical stability; discuss observed empirical scaling versus the theoretical bound.

7 Future Scope

The following extensions are planned for the next phase of the project:

- **MPI (OpenMPI) implementation** — distributing the divide-and-conquer recursion of MAXIMA2/FILTER across multiple processes, with particular attention to load-balanced

merge steps.

- **CUDA implementation** — parallelizing the dominance checks and the sweep/merge steps on the GPU for large point sets.
- **Performance evaluation** — measuring speedup and scale-up of the parallel (MPI/CUDA) versions relative to the sequential baseline, across varying number of points N , dimension d , and number of processes/threads/GPU cores, and comparing strong versus weak scaling behavior for both parallelization strategies.